On the Effectiveness of Set and Multiset Representations in Constraint Programming*

Christopher Jefferson and Alan M. Frisch

AI Group, Department of Computer Science
University of York, UK
[caj,frisch]@cs.york.ac.uk

Abstract. Constraint programming is a powerful and general purpose tool which is used to solve many useful real world problems. The process of transforming an abstract specification of a problem into an efficient CSP (known as modelling) is unfortunately more an art than a science at present and must be learned by years of experience. This paper analyses one reoccurring topic in modelling, how to more effectively represent a set or multiset in a CSP analyses how the choice of representation can effect search.

1 Introduction

When modelling a problem specification as a CSP, there are many choices which must be made. One of these, and arguably one of the most important, is how to represent high level variables such as sets and multisets in the problem specification as one or more CSP variables. For sets [9] and multisets [15], the problem of how to most efficiently implement a specific choice of representation has been well studied. The more general problem of choosing from among a set of representations has been investigated by Hnich [12] for functions, and Hnich et al. [13] for permutations.

The aim of this paper is to expand this work and present a new method by which the representations of sets and multisets can be compared theoretically and how this can be used to aid choice of representation in problems. Also, based upon theoretical weaknesses of some set and multiset models which become apparent when analysing them theoretically, there is a modification of a common model which can overcome some of these problems. While this paper primarily discusses sets and multisets much of the theory discussed is applicable to other commonly occurring constructs.

1.1 Sets and Multisets

Sets and multisets are some of the most commonly occurring high level constructs in both real world and combinatorial problems. Therefore it is important to have efficient methods of representing these in a CSP.

* Many thanks to Ian Miguel for valuable discussions on this work

As will be seen later, there are various classes of set and multiset and the choices of representation and their theoretical properties can alter depending on the constraints imposed on them. To combine the discussions of sets and multisets wherever possible, we shall consider a (multi)set variable to be a variable whose domain is expressed as $MS(X, occ, size)$, which denotes all multisets drawn from X where each element of X occurs at most occ times and the total size is contained in the range $size$. We only consider occ as a single value rather than a range because if there were a non-zero lower bound to the number of occurrences of each value, these could simply be assumed to always occur in the (multi)set and be removed. Three important special cases are the case where occ is 1, which means that the variable is a set variable, where $size$ contains only a single value, so the set is a *fixed size* set, and where $size$ is a sufficiently large range that it does not constrain the domain of the multiset (is at least $[0 \dots |X| * occ]$), which is called an *unsized* (multi)set.

Given a (multi)set S, we shall assume all of the common set constraints and operations have their usual definitions. The only slightly uncommon notation used is $occ(x, S)$, which denotes the number of occurrences of x in S (which will obviously be either 0 or 1 when S is a set). $\mathcal{P}(S)$ represents the set of all subsets of S.

1.2 CSP terminology

A finite CSP is finite set of variables, each with an associated finite domain and a finite set of constraints, each over a subset of the variables. Within this paper it will be assumed all CSPs, sets and other constructs considered are finite.

Given a constraint C, $vars(C)$ is defined to be the set of variables which are contained in C. A constraint can be specified either as the set of tuples over $vars(C)$ which are accepted by the constraint, or in some constraint language.

Given a CSP variable X with domain D, a subdomain of X is a subset of D. Given a vector of variables $\boldsymbol{V}$, a subdomain of $\boldsymbol{V}$ is a vector consisting of a subdomain for each corresponding variable in $\boldsymbol{V}$. At each node during search, the current remaining search space is represented by a subdomain of each variable in the CSP. If any of these subdomains becomes empty then there is clearly no possible solution and therefore the search backtracks.

In this paper it will be assumed that CSPs are solved using 2-way branch-and-propagate search, where at each node in the search tree some variable V is chosen, and its current subdomain split into two non-empty (usually disjoint) subsets, S and T. Two new subproblems are generated, both identical to the original except in one V has domain S and in the other V has domain T. Furthermore, at each node constraints can be *propagated* by removing values from the domains of variables which cannot appear in any solution. It will be assumed that the solver is able only to reason about each constraint in isolation. The best possible level of propagation is GAC propagation, which given a constraint C removes all values from the subdomains of the variables in $vars(C)$ that can never appear in an assignment which satisfies C.

There are various commonly used definitions of symmetry in constraint programming. Within this paper the three which shall be used are *variable* symmetries which are defined as a bijection between variables, *index* symmetries which are defined as a bijection of the index of a vector and *assignment* symmetries which are bijections between all possible assignments to the variables in a CSP. Usually we shall discuss a *symmetry group*, which is a set of symmetries which are closed under composition and inverse. A *complete* symmetry group is the set of all possible symmetries allowed, so for example the *complete variable symmetry group* is the set of all bijections of a set of variables.

A CSP P has a *variable* symmetry group G if given any assignments to the variables of P which is a solution and any permutation $g \in G$, then applying the permutation g to to assignment also gives a solution. A CSP having an *index* or *assignment* symmetry group G is defined similarly.

Given a symmetry group G of a CSP P, then the *orbit* of some assignment to the variables of P is all images of the assignment under G. A set of *symmetry breaking constraints* S on the variables of P is *valid* with respect to G if every assignment has at least one element of its orbit which is allowed by S and is *complete* with respect to G if at most one element of its orbit is allowed by S.

2 Representations

Given a problem which requires finding an assignment to a (multi)set variable with domain $MS(X, occ, size)$, the most direct and obvious method of representing this is as a single CSP variable with domain $MS(X, occ, size)$. This is referred to as the *direct* representation. This is not however the only, or necessarily the most effective, method to use.

Given a variable V with domain D in a CSP, then the usual method of storing the state this variable during search can be any element of $\mathcal{P}(D)$. However it is not necessary to be able to represent all elements of $\mathcal{P}(D)$ to have a correct search. Given some variable V with domain D we must be able to represent the subdomain of V being D (at the start of the search any value is allowed), the subdomain of V being any $d \in D$ (so we can consider any assignment to V) and the subdomain of V being empty (meaning there is no solution in the part of the search).

During search the subdomain of V can be changed in two main ways, by branching (which involves splitting the current subdomain of V into two pieces) and by reducing the subdomain by removing values which it is possible to deduce cannot occur in a solution. If only some elements of $\mathcal{P}(D)$ can be represented it may not be possible to either use a particular branching strategy, or to remove values from the current subdomain of V even when it is possible to deduce they cannot occur in any solution.

As will be seen in Section 3, for some constraints it is possible to categorise the deductions which are possible and which elements of $\mathcal{P}(D)$ can be occur during search. If some given representation can represent each of these elements then it can still perform as small a search as the *direct* representation.

Before alternative representations of sets and multisets in CSPs can be discussed in detail, it is necessary to have a mathematical definition of what it means to represent one variable by one or more other variables, and how such a transformation should be performed. These are defined below.

Definition 1. *A representation of an abstract variable X is a triple $\langle \boldsymbol{V}, f, D\rangle$ where $\boldsymbol{V}$ is a vector of CSP variables, f is a partial injective function from $\boldsymbol{V}$ to X and D is a constraint over $\boldsymbol{V}$ which imposes "$\boldsymbol{V}$ is in the domain of f". An assignment to $\boldsymbol{V}$ which is in the domain of f is said to represent its image under f.*

Above we said that the 3 necessary features of a representation of a variable used in a CSP is that it can represent the complete domain of the variable, each assignment to the variable, and the empty domain. It is simple to see that this representation will satisfies these requirements.

Definition 2. *Given a constraint C and a representation $\langle \boldsymbol{V}, f, D\rangle$ for a variable $X \in vars(C)$, then the constraint C is "replaced by the representation $\langle \boldsymbol{V}, f, D\rangle$" in the following way:*

Consider C as a set of tuples over the vector of variables $\langle X[1], \ldots, X[n]\rangle$ where $X[1]$ is the variable to be replaced by a representation. For each tuple defining C generate a set of tuples for C', over $\langle \boldsymbol{V_1}, X[2], \ldots, X[n]\rangle$, where for each original tuple a set of tuples is constructed, one for each pre-image of $X[1]$ under f, of which there will be at least one, as f is injective. Also D is imposed on $\boldsymbol{V}$.

Definition 3. *Given a constraint C and a mapping from a set S of representations to a subset of $vars(C)$ then "C is represented by S" by the constraints generated by applying all representations in S to C.*

Example 1. Consider a multiset variable X with domain $MS(\{0,1,2,3\},1,2)$. This is *represented* under the *explicit* representation by a triple $\langle \boldsymbol{V}, f, D\rangle$, where $\boldsymbol{V} = \langle V[1], V[2]\rangle$ and the $V[i]$ have domain $\{0,1,2,3\}$, f is defined as mapping each pair of values $\langle V_1, V_2\rangle$ to the set $\{V_1, V_2\}$ where X and Y are distinct, and D is the constraint $V_1 \neq V_2$. Consider the constraint $\sum_{i \in X} = 3$, which when expressed as tuples becomes $\{\langle\{0,3\}\rangle, \langle\{1,2\}\rangle\}$. When this constraint is *refined* via the *representation* $\langle \boldsymbol{V}, f, D\rangle$, then the original constraint is replaced by the constraint $\langle V[1], V[2]\rangle \in \{\langle 0,3\rangle, \langle 3,0\rangle, \langle 1,2\rangle, \langle 2,1\rangle\}$.

Note that while the definition of representation considers constraints as sets of tuples, in general the constraints on the original (multi)set and the new CSP variables can be expressed in a more implicit manner. Automatically refining a list of implicit constraints by a set of representations is part of the more general problem of automatically refining abstract specifications into CSPs and this is explored by other systems, including ESRA [4] and Conjure [6]. This problem will not be explored further here, but instead the new implicit constraint will simply be given. In Example 1 above, the represented constraint could be specified as $V[1] + V[2] = 3$.

Lemma 1. *Given a CSP where a variable X has been replaced by a representation $\langle \boldsymbol{V}, f, D \rangle$ in all constraints where it occurs, then any solution to this new CSP is a solution to the original CSP by applying f to $\boldsymbol{V}$ and leaving all other variables assigned the same values. Also any solution of the original CSP is a solution to the new CSP by any of the inverses of the assignment to X under f as the assignment to $\boldsymbol{V}$ and leaving all other variables assigned the same values.*

Proof. Trivial from definitions. □

Given a representation $\langle \boldsymbol{V}, f, D \rangle$ of an abstract variable X, it is possible to map a subdomain of $\boldsymbol{V}$ (which was defined earlier to be a subdomain of each of the variables in the vector $\boldsymbol{V}$) via f to a subdomain of X by taking each assignment it represents and applying f to it. Given a subdomain of X then in the same way we can take all the assignments to $\boldsymbol{V}$ which map into this subdomain. In general however this will not be exactly representable by the set of assignments to some subdomain of $\boldsymbol{V}$. In this case instead consider the unique smallest subdomain of $\boldsymbol{V}$ which contains all the assignments. This operation will be denoted as f^p to distinguish it from the normal inverse of a function.

2.1 Occurrence representations

One commonly-used method of representing (multi)sets is through a family of representations known as the *occurrence* representation. These are defined as follows:

Definition 4. *Given an abstract variable A with domain $MS(X, occ, size)$, it is represented under the occurrence representation by the triple $\langle \boldsymbol{V}, f, D \rangle$, where $\boldsymbol{V}$ (known as the occurrence vector) is a vector of length $|X|$ of variables with domain $\{0, \ldots, occ\}$, and each element of $\boldsymbol{V}$ is associated with an element of X (so for each x in X we can refer to $V[x]$). The function f maps each assignment of $\boldsymbol{V}$ by taking the sub-(multi)set of X with $V[x]$ occurrences of x for each $x \in X$, where these (multi)sets lie in $MS(X, occ, size)$. D is defined as $\sum_{x \in \boldsymbol{X}} \boldsymbol{V}[x] \in size$.*

The most important special case of the *occurrence* representation is where the (multi)set is *unsized*, and therefore the sum constraint is redundant. This can make a difference to the theoretical effectiveness of the representation, as shall be shown later.

2.2 Explicit representations

The other major category of representations of sets and multisets is *explicit* representations. These are constructed as follows:

Definition 5. *Given an abstract variable A with domain $MS(X, occ, size)$, it is represented under the explicit representation by $\langle \boldsymbol{E}.\boldsymbol{C}, f, D \rangle$, where $\boldsymbol{E}$ and $\boldsymbol{C}$ are both of length $max(size)$, the elements of $\boldsymbol{C}$ are boolean variables and the elements of $\boldsymbol{E}$ have domain X. The function f maps assignments of $\boldsymbol{E}$ and $\boldsymbol{C}$*

to the (multi)set which contains each $E[i]$ for which $C[i]$ is TRUE*, where this (multi)set is in $MS(X, occ, size)$. $\boldsymbol{E}$ is called the element vector, and $\boldsymbol{C}$ is called the check vector.*

D constrains $\boldsymbol{E}$ and $\boldsymbol{C}$ so that $\sum_{c \in \boldsymbol{C}} \in size$ and whenever $occ + 1$ elements of $\boldsymbol{C}$ are TRUE*, then not all the corresponding elements of $\boldsymbol{E}$ can equal.*

In the case where the *explicit* representation represents a fixed size (multi)set, all variables in the *check* vector are constrained to be TRUE and this can greatly simplify problems in some solvers when applied to the constraints before search begins.

There are two main problems with the *explicit* representation as it is given above, both relating to the case that any particular (multi)set may be represented by multiple assignments to the *explicit* representation. The first is that when the i^{th} element of the *check* vector is FALSE, then the value of the i^{th} element of the *element* vector is never used. However all possible assignments to this variable may still be searched. This is dealt by altering f (and therefore D) defined in Definition 5 to constrain the *element* vector to take some specific value when the *check* vector is FALSE, which for reasons which will become clear shortly will here always be the largest value in the domain of the *element* vector. In general, a better method may be to remove the *check* vector and introduce a value into the domain of the *explicit* vector which represents this variable not being in the vector. While this would simplify the *explicit* representation, this would make it more difficult to use with existing CSP solvers and constraints which are not designed to cope with this kind of "distinguished element" in the domain of a variable.

This however still leaves cases where there are multiple assignments to the *explicit* representation which represent the same (multi)set. There is *variable* symmetry in the representation, as given an assignment to the *element* and *check* vector, then both vectors are permuted in the same manner then the resulting vectors will represent exactly the same (multi)set. One of the simplest and most frequently used methods of dealing with this symmetry is to redefine the representation so that the only assignments which are mapped to the abstract variable are those where the *check* vector is sorted largest to smallest (and therefore consists of a sequence of 1s followed by a sequence of 0s), and where the *check* vector is 1, the *element* vector is ordered smallest to largest. This will clearly break all of the variable symmetry, as for each (multi)set this will allow exactly one assignment to the *check* and *element* vectors which represents it. As we earlier imposed that when the i^{th} element of the *check* vector is 0 the *element* vector will take its largest value, this can be simplified to force the *check* vector to be non-increasing and the *element* vector to be non-decreasing.

This can be improved further by noting that if the maximum number of occurrences of each value is *occ*, then the (i+*occ*)th element of the *element* must be strictly larger than the ith element when all elements of the *check* vector between i and $i + occ$ are 1.

3 Perfect Representations

When a choice of how to represent some particular variable exists, then it would obviously be useful to have method to enable choosing between them. The best method would find the representation which allowed for one or all solutions to a given problem to be found in the smallest time. This is unfortunately a difficult goal for most problems, as we cannot know the exact idiosyncrasies of each individual solver and computer, and also for large problems the interaction of the various parts of the search procedure are not fully understood.

One previous successful theoretical comparison of alternative representations was performed on permutations [13]. One of the reasons for this success is that the representations studied there were closely related and given a set of subdomains of the variables in one representation, it was always possible to represent exactly the same assignments using a set of subdomains in the other representations considered. This does not hold true for the representations of sets and multisets considered here, so different methods of comparing representations must be found.

As was discussed in section 1.2, the current state of the search at each node in the search tree is stored as a subdomain of each of the variables. Therefore one possible metric for comparing representations is to examine which subdomains of the original variable they are able to represent. When a representation cannot represent some subdomain of the original variable which could arise during search, it has to represent some larger subset of the original variable. This "loss of representation power" can lead to larger search trees, as will be seen later. It is obvious that the *direct* representation of a set or multiset will be able to represent all possible subdomains, and unfortunately both the *occurrence* and *explicit* cannot represent many subdomains.

Lemma 2. *Given the set variable A with domain $MS(\{1,2,3\},1,[0,\ldots,3])$, the occurrence representation cannot exactly represent the subset of its domain $\{1\},\{2\}$, and the explicit representation cannot exactly represent $\{1,2\},\{1,3\},\{2\}$.*

Proof. The occurrence representation of A have an *occurrence* vector of 3 boolean variables. In order to represent $\{1\},\{2\}$ these variables must have at least the values $\langle\{0,1\},\{0,1\},0\rangle$ in their domain. These subdomains will also represent the sets $\{\}$ and $\{1,2\}$.

An *explicit* representation of $\{1,2\},\{1,3\},\{2\}$ must have one variable with 1 in its domain. If this element of the *element* vector must be in the set as the appropriate value in the *check* vector has been assigned 1, then this variable must also be able to take the value 2, as we must be able to represent just the set $\{2\}$. In this case some other variable must be able to take the value 3, so the variables can be assigned the set $\{2,3\}$. If this variable does not have to be in the final set, then some other variable must be able to take the value 3, and so the set $\{3\}$ is represented. □

Lemma 2 shows that in terms of representing possible states of a multi(set) variable, the *direct* representation is strictly stronger than either the *explicit* or

occurrence representations. This obviously leads to the question of why to use any representation other than the *direct* one.

The primary reason for this is that in the worst case the *direct* representation requires a huge amount of data to be stored at each node of the search tree. Consider the example of a variable A with domain $MS(\{1,\ldots,n\},1,[0\ldots n])$. A has a domain of size 2^n, and therefore there are 2^{2^n} possible subdomains of A, so in the worst case A will require 2^n bits of memory to store its state. In many problems this is too large to be feasible, and in cases where this much information can be stored, the added amount of time taken to maintain this size of domain slows the solver down so much that other representations perform a faster if larger search.

While the worst case storage of the *direct* representation is exponential, it is not necessary for the representation to be fixed size. For example Lagoon and Stuckey [14] investigate using reduced ordered binary decision diagrams (ROBDDs) to represent subdomains of the *direct* representation. While in the worst case these can grow to exponential size, in experiments they manage to remain at a reasonably small size. As expected they lead to smaller search spaces than *occurrence* and *explicit* representations, sometimes by a huge amount. However the worst case is still exponential, and for some problems the extra processing per node may be outweighed by the reduced search tree. Therefore it is still worth investigating other representations.

A second reason however is that the gains from using the *direct* representation in some problems may be small, or even non-existent. While the *explicit* and *occurrence* representations cannot represent all possible subdomains of the *direct* representation, as for some particular search strategy and set of constraints they may be able to represent all subdomains of the original variable which occur.

As was discussed in section 1.2, the best propagation a solver can achieve on a single constraint is *GAC*. In a similar fashion, there is a best level of performance a representation can achieve with respect to a particular constraint, where it is able to represent all subsets of the domain of the variable it represents which occur during search.

Definition 6. *Given a constraint C, a set S of representations for C and the constraint C' obtained by applying them to C, a specific representation $R = \langle \boldsymbol{V}_x, f_x, D_x\rangle$ from S, then "R is perfect with respect to C and S" if for any subdomain of $vars(C')$ which are GAC, its image under f_x is GAC.*

Note that the definition of *perfect* requires both the constraint and a set of representations, not just one. This is because replacing a single variable in a constraint with a representation may not be perfect, while replacing the same variable with a representation while simultaneously other variables is.

Example 2. Consider a variable A with domain $MS(\{1,\ldots,n\},1,[0\ldots n))$ represented with the *occurrence* representation with *occurrence* vector $\boldsymbol{V}$. The constraint $a \in S$ for a constant value a when mapped to the representation is represented by the constraint $V[a] = 1$. Any subdomain of $\boldsymbol{V}$ which is *GAC* with respect to this constraint will clearly have $V[a]$ with only 1 in its domain.

It is obvious therefore that any assignment to $\boldsymbol{V}$ which can be mapped to A will satisfy the original constraint. Therefore the *occurrence* rep is perfect with respect to the constraint $a \in S$.

Example 3. Given a natural number n, consider the CSP containing a single set variable A with domain $MS(\{1,\ldots,2n\},1,[0\ldots 2n])$ and the two constraints $|A| = n$ and $|A| = n+1$. If S is left as the *direct* representation then any solver which can achieve GAC propagation for these two constraints will stop without any search, as no element in the domain of A will satisfy both constraints. Now consider S being represented by an *occurrence* representation with *occurrence* vector V. On any subdomain of V where less than $n-1$ variables are instantiated, then clearly of the remaining variables at least one must be assigned 1 and at least one must be assigned 0. Also, there will be an instantiation of the variables where the sum is $n-1$, and there will be one where the sum is n. As this is the case, the search can neither fail at this node, nor assign any variables, so will continue. This means that no branch of the search can either assign variables through propagation or fail before a depth of $n-2$ (and in some places may have to search much deeper than this), and therefore the size of the search tree will be at least 2^{n-2}.

Example 3 demonstrates how using representations can lead to exponential growth in the size of the search tree. This is not the case when a *perfect* representation is used, as at each node of the search tree then after GAC is achieved, there are no assignments allowed by the representation which would not be allowed by the original variable.

Definition 7. *Given a constraint C and a set of constraints S, then S is a split of C if $vars(C) = \cup\{vars(s)|s \in S\}$ and C is logically equivalent to $\wedge\{s|s \in S\}$.*

Lemma 3. *Given a split S of a constraint C, then the set of constraints S' generated by creating an $s' \in S'$ for each $s \in S$ where $vars(s') = vars(s)$ and an assignment is allowed by s' if it can be extended to a valid assignment of C is also a split of C.*

Proof. There cannot be an assignment to $vars(C)$ which is allowed by C and not by $\wedge\{s'|s' \in S'\}$ as the $s' \in S'$ are defined so that they will allow any assignment which can be extended to a valid assignment of C. There cannot be an assignment to $vars(C)$ which is not allowed by C but is allowed by $\wedge\{s'|s' \in S'\}$ as the definition of the $s' \in S'$ disallows all the tuples which can possibly be disallowed, therefore $\wedge\{s|s \in S\}$ must allow at least as many assignments as $\wedge\{s'|s \in S'\}$ and any assignment disallowed by C is disallowed by S. □

Theorem 1. *Given a constraint C, a representation C' of C generated by applying a set of representations R to C, then a representation $\langle \boldsymbol{V_x}, f_x, D_x\rangle \in R$ is perfect with respect to C and R if and only if C' can be split into a set of constraints S, each of which contains at most one variable from $\boldsymbol{V_X}$.*

Proof. ($\rightarrow$) If there exists a valid split of C' into a set of constraints S which each contain at most one variable from $\boldsymbol{V_x}$, then there will exist a set of constraints

S which split C' which consist of exactly one constraint for each element of $\boldsymbol{V_x}$ which each have one variable of $\boldsymbol{V_x}$ and all variables in $vars(C') - \boldsymbol{V_X}$. Further if such a split exists we know exactly the form the constraints will take from Lemma 3. The constraints as defined in 3 cannot forbid allowed assignments, so it must be checked if they can allow assignments forbidden by C'. Consider an assignment A to $vars(C')$ which is forbidden by C' and allowed by $\wedge\{s|s \in S\}$. This would mean that for each $v \in \boldsymbol{V_x}$ there is an assignment A_v which agrees with A on the assignment of v and $vars(C') - \boldsymbol{V_x}$. If we take the union of these assignments, the resulting subdomains will satisfy GAC, as obviously all values in each variable are part of a valid assignment to the variables. The variables in $vars(C') - \boldsymbol{V_x}$ will each only have one value in their domains.

Finally, as the constraint is perfect we know that GAC implies that any assignment to $\boldsymbol{V_x}$ can be extended to a valid assignment. Therefore assigning $\boldsymbol{V_x}$ the values in the assignment A must be extensible to a valid assignment. There is only one possible extension however, which is A itself.

($\leftarrow$). Assume that C' can be *split* into a set of constraints which each contain at most one variable from $\boldsymbol{V_X}$. Assign all the variables in $vars(C) - \boldsymbol{V_X}$ some fixed value and then consider all assignments to the variables in $\boldsymbol{V_X}$ which satisfy all the constraints in S. Apart from the constraint "$\boldsymbol{V_X}$ has an image under f_X", all the other constraints refer to only a single variable in $\boldsymbol{V_X}$ and therefore the assignments to these variable are independent (assuming they have an image under $\boldsymbol{V_X}$) and therefore the constraint is *perfect*. □

Theorem 1 provides a method of telling if a constraint will be perfect. For example consider any constraint defined over a set of variables each with domain $MS(X, occ, [0, occ \times |X|])$ built using the operators $\cup, \cap, \subset, =$. It is simple to see that such a constraint can be split into a series of constraints each of which refers to only one element in X, for example $A \cap B = C$ is logically equivalent to a set of constraints $\{(x \in A) \wedge (x \in B) \leftrightarrow (x \in C)|x \in X\}$. When this constraint is represented using the *occurrence* representation then $x \in A$ will become "the x^{th} element of the *occurrence* vector is not 0", and therefore each of the these constraints will refer to only one variable in the *occurrence* vector of the representations of A,B and C, and therefore the representation will be *perfect*.

Unfortunately, the *explicit* representation is not *perfect* for any of the commonly used constraints over sets, including those built from $\neq, \cup, \cap$ and $\subseteq$. This is simple to see, as it is impossible to decide if a specific value is within a (multiset represented by the *explicit* representation without looking at all its elements.

3.1 The explicit subset representation

The explicit representation with *static* symmetry breaking imposed on the *element* vector, as defined in Section 2.2 suffers from a common problem associated with static symmetry breaking, which is that the choice of value and variable ordering more strongly effect search after the symmetry breaking constraints have been imposed. Now, setting the i^{th} element of the *element* vector to be j when the i^{th} element of the *check* vector is 1 does not only impose that j will be in the

(multi)set, but more specifically imposes that j will be the i^{th} largest element. There may exist problems this kind of more specific ordering knowledge is useful, but in general can leads to an increase in search size, as when this decision is backtracked over the only new information is that the i^{th} largest element is not j, which is quite a weak deduction. Usually however the gains from symmetry breaking outweigh any loses.

Related to this is one of the major weaknesses of the *explicit* representation. Consider for example the constraint $A \cap B = C$ over three sets A, B and C. If during search it is possible to deduce that A will not contain some value x, then obviously x will also not occur in C. If C is represented by the *explicit* representation, this is not problem as x can simply be removed from the domain of each variable in the *element* vector. If however assignments were made so that x was in both A and B, then while it is now apparent that x will be in the set C, it may not be possible to represent this. Clearly in a solution, one value in the *element* will be assigned x, but if they are ordered there is no way of knowing which one unless x is either the smallest or largest remaining value (in which case it will be the right-most or left-most uninstantiated variable respectively). In the worse case, this can lead to a situation where some node of a search tree obviously has no solution as the number of values that C must take are greater than its maximum size, but none of the *element* vector of C has yet been assigned the CSP solver does not detect this. It is possible that this problem could be reduced or removed using dynamic symmetry breaking and assigning these values as search progresses, but such systems have not been fully investigated yet.

Therefore the *explicit* representation cannot be perfect on common (multi)set constraints such as $A \cap B = C$ and $A \subset B$. This problem can however be alleviated somewhat by an adaption of the *explicit* representation which makes more use of the structure of the constraints.

Normally a (multi)set variable A is represented under the *explicit* representation by two vectors, the *element* vector and the *check* vector. Therefore given the constraint $A \subset B$, both A and B will have separate *element* and *check* vectors. There is however an alternative. As $A \subset B$, then clearly any element of A will also be an element of B. Therefore the same *element* vector can be used for both A and B (note that the elements of A will probably not all be at the start of B's *element* vector so it is necessary to remove the ordering on A's *check* vector). If A's *check* vector is V and B's is W, then the constraint $A \subset B$ becomes $\forall i \in [1 \dots n].V[i] \rightarrow W[i]$, and by an almost identical proof that $A \subset B$ was perfect under the *occurrence* representation, this constraint is also *perfect* in this representation. This method can also be used on union constraints, as given the constraint $A_1 \cup \dots \cup A_n = B$, we know that $A_i \subset B$ for all i, and so the *explicit* representations of the A_i can use the same *element* as B. Following the same proof as the *occurrence* representation of this constraint being perfect, so is this one.

This method is not however a fix for all problems involving the explicit representation. In particular this does not allow for making the constraint $A \cap B = C$

perfect, as this new representation allows for the *explicit* representation of C to use either A's or B's *element* vector, but it cannot use both.

4 Nested constraints

A CSP will often contain large and complex constraints. One way in which these can be handled, both in terms of implementation and theoretical examination is by splitting them into smaller constraints by the introduce of extra variables.

Example 4. Given the constraint $(A \cup B) \cap (C \cup D) = E$ over set variables A, B, C, D, E, this can be decomposed into simpler constraints by the introduction of two new variables X and Y and replacing the original constraint $A \cup B = X, C \cup D = Y, X \cap Y = E$

There are three reasons to be interested in studying complex constraints in this manner. Firstly if all constraints can be split into a finite set of constraints, then by studying only this set and also the process of splitting constraints, all constraints can be considered. Secondly, the method by which complex constraints are often implemented in CSP solvers is by decomposing complex constraints, so studying this is also useful to analyse the performance of constraints in real solvers. Finally, if some expression is contained in a number of constraints then decomposing constraints can lead to a smaller search space. Examples of each of these concepts has been studied in the specific case of arithmetic constraints by Harvey and Stuckey [11], and this is expanded here to work on arbitrary constraints.

Definition 8. *Given a constraint C, a function $f : \boldsymbol{V} \to \boldsymbol{X}$ where $\boldsymbol{V} \subseteq vars(C)$ and a constraint C' with $vars(C') \subseteq vars(C) + \boldsymbol{X}$ such that an assignment to $vars(C)$ satisfies C if and only the same assignment with $f(\boldsymbol{V})$ added satisfies $vars(C')$ then C is "decomposed" by adding the variables $\boldsymbol{X}$ and replacing C with C' and the constraint $f(\boldsymbol{V}) = \boldsymbol{X}$. If $vars(C') \subseteq vars(C) + \boldsymbol{X} - \boldsymbol{V}$, the decomposition is pure.*

Example 5. Given the constraints $A + B + C = 0$ and $A + B + C = 1$ where A, B and C have domain $\{-1, 0, 1\}$, it is possible to *decompose* both of these constraints by adding the new variable X with domain $\{-2, \ldots, 2\}$, adding the constraint $A + B = X$ and replacing the two original constraints with $X + C = 0$ and $X + C = 1$.

Example 6. Given the constraint $A + B * A = 0$ where $A \in \{-n, \ldots, n\}$ and $B \in \{-1, 1\}$, then this constraint could be *decomposed* by introducing a new variable X with domain $\{-n, \ldots, n\}$ where X is defined $B * A$. This would produce the new problem with the original constraint replaced by $A + X = 0$ and $B * A = X$. This *decomposition* is not *pure* as A occurs in both constraints, and it is possible to show that while A must be assigned 0 to satisfy the original constraint, but with their default domains both of the two new constraints are *GAC*.

While replacing a constraint with a *decomposed* constraint is always a valid operation, it does not necessarily lead to an identical search space. One obvious reason for this is that a number of extra variables have been introduced. This problem can be avoided by having heuristics ignore any such variables and leaving them until all other variables have been assigned. When all other variables have been assigned then an assignment will be forced on the introduced variables, as each introduced variable X is contained in a constraint of the type $f(\boldsymbol{V}) = X$ for some function f and some vector of variables V. This may not however be the best choice, as branching on the newly introduced variables may allow for a smaller search.

Apart from this, the other obvious way in which *decomposed* constraints can effect search is by removing possible propagation, as in example 6. This particular kind of loss can occur whenever the *decomposition* is not *pure*. Even when given a pure *decomposition*, loss of propagation can occur when the domain of f is a vector of variables.

Theorem 2. *Consider a constraint C which can be pure decomposed using a function $f : \langle \boldsymbol{V} \rightarrow \boldsymbol{X}$ and constraint C' (we shall refer to the constraint $f(\boldsymbol{V}) = \boldsymbol{X}$ as C_f). Assume every assignment to $\boldsymbol{X}$ which is not in the domain of f never appears in a valid assignment of C_f or C' and given any subdomain of either $vars(C_f)$ or $vars(C)$ which is GAC, then an assignment to $\boldsymbol{X}$ both contained in the subdomain and in the domain of f can always be extended to a valid assignment.*

In this case, given a subdomain D of $vars(C)$, if D fails to satisfy $GAC(C)$ then any extension of D will fail to satisfy both $GAC(C_f \wedge C')$ and $GAC(C_f) \wedge GAC(C')$ no matter what domain the variables in $\boldsymbol{X}$ are given.

Proof. a) Clearly $GAC(C_f \wedge C') \rightarrow GAC(C_f) \wedge GAC(C')$. Therefore consider subdomains of each of the $vars(C)$ and $\boldsymbol{X}$ such that $GAC(C_f) \wedge GAC(C')$ is satisfied, but $GAC(C)$ is not. There must therefore exist $s \in vars(C)$ and assignment a to s which cannot be extended to a valid assignment of C. There are two cases to consider. For $s \in vars(C) - \boldsymbol{V}$ then as $GAC(C_f)$ there must be an assignment to $\boldsymbol{V}$ and $\boldsymbol{X}$ with $s = a$ which satisfies C_f. As the assignment contained in a to $\boldsymbol{X}$ is part of a valid assignment to C_f, then it must be in S, and therefore it must be extendable to an assignment of C'. This assignment therefore satisfies C. The case where $s \in \boldsymbol{V}$ follows similarly. □

One important point to note about the precondition in theorem 2 involving a set of assignments is that in the case where the function f returns only a single variable this is trivially true (as it is equivalent to the definition of GAC), and more importantly, in the case where a variable is replaced by a representation which is perfect in both of the two new constraints, then the set of variables generated will satisfy this condition where the set required by the theorem is the set of assignments which represent assignments to the high level object.

Example 7. Given the constraint $(A \cup B) \cap (C \cup D) = E$ where A, B, C, D, E are all variables with domain $MS(\{1, \ldots, n\}, 1, [0 \ldots \infty))$, then this can be decomposed into $A \cup B = X$, $C \cup D = Y$ and $X \cap Y = E$ for new sets X and

Y with the same domain, and if all the sets are represented via the occurrence representation then this has not lead to a loss of propagation.

5 Recursive sets and multisets

So far we have considered variables with domain $MS(X, occ, size)$ without paying attention to any structure that X may have. There are many problems which involve finding a (multi)set of (multi)sets, for example the Steel Mill [7] problem requires finding a set of sets under a various capacity constraints, and even more complex the Social Golfers Problem [10] requires finding a set of set of sets. So far we have only discussed representing a set of some unknown type with the *occurrence* or *explicit* representation. In the case where these more complex types exist, then this may be insufficient.

Example 8. Consider the main variable of the BIBD problem, which is a variable of domain $MS(X, b, b)$ where $X = MS(S, 1, a)$ for some set S and constants a and b.

Under the *occurrence* representation, this would generate a single vector of variables, each with domain b. The length of this vector would be $|MS(S, 1, a)|$. Under the *explicit* representation, this would be represented by an *element* vector of length b, where each variable had domain $MS(S, 1, a)$.

In Example 8, the *occurrence* representation leaves little possibility for further replacement of variables by representations. The *explicit* representation on the other hand generates a vector of variables with domain $MS(S, 1, a)$, which are themselves sets and could therefore be replaced by a representation. This demonstrates the single largest difference between the *occurrence* representations (which tend to refine any object to a final form without allowing for further refinement) and the *explicit* representations (which allow in some cases for further refinement).

The existing theory of representations is clearly able to cope with recursive applications of representations, as once a representation has been applied then the result is another CSP, which can of course have further variables replaced with representations. Similarly, Section 3 discussed how replacing a constraint with a *perfect* representation does not increase the size of the search space, and this can obviously be applied recursively.

One of the most important differences about recursive (multi)sets is that in the same way that the *direct* representation can be too large for use in many common problems, in the common case of searching for a set of small arity drawn from some much larger set (as in the Example 8), the occurrence representation can grow very large while the *explicit* representation remains much smaller. Therefore despite the fact that for many constraints the *explicit* representation can perform much more poorly, for the same reason the *direct* representation had to be abandoned the *occurrence* representation must be abandoned for many problems with recursive (multi)sets.

6 Recursive Symmetry Breaking

Symmetry in CSPs can be present in both the original problem specification, or have been introduced during the modelling process, for example as happens in the *explicit* representation. If this symmetry is simply left in the problem, then when searching large symmetric areas of search tree can arise and therefore there have been methods developed to try to reduce this redundant search. The first type, known as *static* symmetry breaking, was discussed briefly in section 1.2. A second commonly used method, called *dynamic* symmetry breaking, involves altering the search procedure itself so that whenever a search state is encountered which is symmetric to one which has been previously searched, then it is not explored. Systems which implement this idea include SBDS [8] and SBDD [2].

Static symmetry breaking has been explored in depth both in theory [1] and in practice [5, 3]. The methods defined in these papers assume that the basic objects on which they operate are integer variables. While it shall be stated here without proof, in general they only require that the objects on which they operate are have a total ordering defined on them.

In Example 8, a set of sets were represented. These could be represented by an *explicit* representation of *explicitly* represented sets. As in this case all the sets have fixed size the *check* vectors can be removed, and therefore the set of sets would be represented by a vector of vectors. As a simple implementation of the *explicit* representation has *variable* symmetry, this means that the variables in each of the inner vectors the variables can be permuted, as can the index of the outer vector.

Normally, both *static* and *dynamic* symmetry breaking methods work upon a single symmetry group which represents the symmetries of the whole problem under consideration. However in many problems the symmetry groups can naturally lie in a hierarchy, and it is possible to make use of this.

Theorem 3. *Consider a vector $\boldsymbol{V}$ of identical representations where each element of $\boldsymbol{V}$ has assignment symmetry group G_A and $\boldsymbol{V}$ has index symmetry G_V. Given a set C_A of symmetry breaking constraints for G_A and a set C_V for G_V, then imposing C_A on $\boldsymbol{V}$ and C_V on each element of $\boldsymbol{V}$ is valid if C_A and C_V are and complete if C_A and C_V are complete.*

Proof. Consider imposing C_V on all elements of $\boldsymbol{V}$. This is valid/ complete if C_V is valid/complete. The result of this is still a vector of symmetric objects and therefore will still have symmetry group G_A. Therefore applying C_A to $\boldsymbol{V}$ will by valid/complete on V.

One of the major advantages of using this method is that the number of constraints required can be greatly reduced. For example the *Crawford*[1] method of symmetry breaking requires n constraints for a group of size n. In the case of a vector $\boldsymbol{V}$ where the vector has *index* symmetry group G_A and each element has a *variable* symmetry group G_V then the size of the combined group and there the number of required constraints is $|G_A| * |G_V|^n$. In the case where the symmetries on $\boldsymbol{V}$ and its elements are broken separately only $|G_A| + |G_V| * n$ constraints are required.

We shall not consider how to combine dynamic symmetry breaking methods, but shall consider the possibility of taking a vector of representations V and either applying *dynamic* symmetry breaking to either V or its elements and *static* symmetry breaking to the other.

It is obviously valid to apply *dynamic* symmetry breaking to an *index* symmetry of $\boldsymbol{V}$ and *static* symmetry to its elements, as after applying the *static* symmetry breaking the resulting CSP still has the same *index* symmetry as before. The reverse is not true.

Theorem 4. *Consider a vector of representations V where V has index symmetry group G_I and each element of V has assignment symmetry G_V. Then applying dynamic symmetry breaking to the elements of V and static symmetry breaking to V is in general may not valid.*

Proof. Consider a vector of vector of variables V, where V has two elements, each a vector of 2 elements, where both V and its elements have complete *index* symmetry. If $V[a, b]$ refers to the b^{th} element of the a^{th} element of V, then a valid symmetry breaking constraint for *index* symmetry of V is $V[1, 1]V[1, 2] \leq_{lex} V[2, 1]V[2, 2]$.

Consider that $V[1, 1] = 1, V[1, 2] = 2, V[2, 1] = 2, V[2, 2] = 1$ and its symmetric equivalents are solutions. If the first symmetric assignment to this which arises during search is $V[1, 1] = 2, V[1, 2] = 1, V[2, 1] = 1, V[2, 2] = 2$, then this will be disallowed as a solution by the *static* symmetry breaking. However after this the dynamic symmetry breaking will now forbid any assignment which can be obtained by permutations of $V[1, 1], V[1, 2]$ and $V[2, 1], V[2, 2]$, which covers are symmetric equivalents of the solution. □

7 Future Directions

This paper has presented a theoretical investigation as to the properties of set and multiset representations, but has also opened a large number of new questions and left a number of open problems. There are three main areas in which we intend to extend this work. Firstly while the discussion of *perfect* representations demonstrates how the representational power of constraints can be compared, it has not presented a complete categorisation, and also has not covered some variants of the *occurrence* and *explicit* representations, in particular adding a variable to the *occurrence* representation which records the size of the set. While this obviously makes the constraint constraining the size of the (multi)set *perfect*, it perhaps surprisingly removes *perfectness* from all the other constraints which were discussed in section 3. This is an area which requires further investigation.

Other obvious ways to explore and expand the ideas in this paper are to further explore the combination of static and dynamic symmetry breaking methods and their applicability to the layered symmetries discussed in Section 5, which occur frequently when refining large objects in multiple stages. Further, many of the ideas discussed here do not apply only to sets and multisets, and could be extended to other abstract types including functions, relations and graphs.

8 Conclusion

This paper has produced a new method of comparing representations of single variables with respect to constraints upon them and show that a family of representations satisfies these conditions. While this paper has not produced a complete categorisation of the representations of sets and multisets it has shown that analysing and comparing very different representations is possible and can produce in some cases very strong results about the resulting searches.

References

1. J. Crawford. A theoretical analysis of reasoning by symmetry in first-order logic. In *AAAI Workshop on Tractable Reasoning*, 1992.
2. T. Fahle, S. Schamberger, and M. Sellman. Symmetry breaking. In *7th International Conference on the Principle and Practice of Constraint Programming (CP)*, pages 93–107, 2001.
3. P. Flener, A. M. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, and T. Walsh. Exploiting common patterns in constraint programming. In *International Workshop on Reformulating Constraint Satisfaction Problems*, 2002.
4. P. Flener, J. Pearson, and M. Agren. Introducing ESRA, a relational language for modelling combinatorial problems. In *Proceedings of LOPSTR '03: Revised Selected Papers*, volume 3018 of *LNCS*, 2003.
5. A. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, and T. Walsh. Multiset ordering constraints. In *18th International Conference in AI*, 2003.
6. A. Frisch, C. Jefferson, B. Martínez Herández, and I. Miguel. The rules of constraint modelling. Technical report, APES 85, 2004.
7. A. M. Frisch, I. Miguel, and T. Walsh. Symmetry and implied constraints in the steel mill slab design problem.
8. I. Gent and B. Smith. Symmetry breaking during search in constraint programming. In *Proceedings of the European Conference on Artificial Intelligence*, pages 599–603, 2000.
9. C. Gervet. Conjunto: constraint logic programming with finite set domains. In M. Bruynooghe, editor, *Logic Programming - Proceedings of the 1994 International Symposium*, pages 339–358, Massachusetts Institute of Technology, 1994. The MIT Press.
10. W. Harvey. Symmetry breaking and the social golfer problem. In *Proceedings of SymCon'01: Symmetry in Constraints*, pages 9–16, 2001.
11. W. Harvey and P. Stuckey. Improving linear constraint propagation by changing constraint representation. *Constraints*, 8[2]:173–207, 2003.
12. B. Hnich. *Function Variables for Constraint Programming.* PhD thesis, Uppsala, 2004.
13. B. Hnich, B. Smith, and T. Walsh. Dual modelling of permutation and injection problems. *Journal of Artificial Intelligence Research, Volume 21*, 2004.
14. V. Lagoon and P. J. Stuckey. Set domain propagation using ROBDDs. In *Proceedings of CP04*, 2004.
15. T. Walsh. Consistency and propagation with multiset constraints: A formal viewpoint. In *Proceedings of CP*, 2003.